ViSPACE User Manual

Virchow-based Spatial Characterization and feature Extraction

Himangi Srivastava

August 4, 2026

Contents

Chapter 1

Installation

This chapter describes how to install ViSPACE, authenticate with Hugging Face, verify the software environment, and prepare the pipeline for whole-slide image analysis.

1.1 Conda Installation

ViSPACE is designed for Python 3.11. Miniforge or another Conda distribution configured to use `conda-forge` is recommended.

Option A — install from source *(recommended)

```
# Create a new environment
conda create -n vispace python=3.11 -y

# Activate it
conda activate vispace

# Clone the repository
git clone https://github.com/himangi2003/ViSpace.git
cd ViSpace

# Install the matching PyTorch build
pip install torch==2.5.1+cu121 torchvision==0.20.1+cu121 \
    --extra-index-url https://download.pytorch.org/whl/cu121

# Install the package and its dependencies
pip install .
```

Option B — Conda environment

```
git clone https://github.com/himangi2003/ViSPACE.git
cd ViSPACE

conda env create -f environment.yml
conda activate vispace
pip install .
```

The environment installs `mussel-pathology[torch-gpu]==1.4.4`; a separate Mussel repository checkout and `MUSSEL_DIR` setting are not required.

1.2 Google Colab

Google Colab does not use the local Conda environment. Install the pip-only requirements instead:

```
pip install -q -r requirements/colab.txt
pip install -q .
```

Colab already provides a notebook runtime. Review the installed PyTorch and CUDA versions before replacing Colab's preinstalled PyTorch build.

1.3 Hugging Face Authentication

ViSPACE uses the gated Virchow2 model. Authentication is required when `VIRCHOW2_PATH=None`; it is not required when local Virchow2 weights are provided.

1.3.1 Account, token, and model access

1. Create and verify an account at https://huggingface.co.
2. Create a read token under **Settings** → **Access Tokens**.
3. Request access at https://huggingface.co/paige-ai/Virchow2.
4. Never commit the token to Git or publish it in a notebook.

1.3.2 Command-line login

```
huggingface-cli login
huggingface-cli whoami
```

1.3.3 Notebook login

```
import os
from huggingface_hub import whoami

HF_ACCESS_TOKEN = "hf_your_token_here"
user = whoami(token=HF_ACCESS_TOKEN)
print(f"Authenticated as: {user['name']}")
os.environ["HF_TOKEN"] = HF_ACCESS_TOKEN
```

Clear notebook outputs and replace the token with a placeholder before sharing.

1.4 Environment Check

Run the pre-flight check before a long analysis:

```
python environment_check.py --from-json run_config.json
```

The checker validates Python, required packages, the Mussel package version, PyTorch and CUDA availability, Hugging Face access, input paths, the checkpoint, and output-directory permissions.

Chapter 2

Configuration

All ViSPACE parameters are defined in a single configuration file, `config.py`. The configuration is implemented through the `PipelineConfig` dataclass and is shared by every stage of the pipeline, ensuring that all components operate using a consistent set of parameters. This design centralises the configuration of the complete workflow, from whole-slide image tessellation to spatial feature extraction, while avoiding duplicated command-line arguments.

2.1 Required Parameters

All settings are defined by the `PipelineConfig` dataclass in `config.py`. Table 2.1 is the single authoritative configuration reference for the pipeline. Parameters marked **Fixed** should remain unchanged for the released ViSPACE model. Parameters marked **Adjustable** may be tuned for hardware or dataset characteristics. Parameters marked **Optional** affect optional outputs or advanced filtering.

2.2 Tesselation Parameters

The first stage of the ViSPACE pipeline performs tessellation of the whole-slide image using the Mussel framework. Tissue regions are identified before the slide is divided into fixed-size image patches, which form the input to the segmentation model. For routine analyses, the default tessellation parameters are recommended. Users may adjust `WORKERS`, `SEGMENT_THRESH`, and `THUMBNAIL_SIZE` according to their computational resources or image characteristics. However, `PATCH_SIZE` and `MPP` should remain fixed at **224 pixels** and **0.25 μm/pixel**, respectively, to maintain compatibility with the pretrained ViSPACE segmentation model and ensure consistent spatial measurements throughout the pipeline.

2.3 Model and Inference Parameters

The parameters in this section control loading of the pretrained ViSPACE segmentation model and the execution of inference. These settings determine how the model is initialised and how image patches are processed during segmentation.The default model parameters are suitable for most analyses. Users with limited GPU memory may reduce `BATCH_SIZE` to avoid out-of-memory errors, while systems without a compatible NVIDIA GPU may set `DEVICE` to `cpu`. If `VIRCHOW2_PATH` is left as `None`, the Virchow2 encoder weights are downloaded automatically from the Hugging Face Hub after successful authentication.

2.4 Stitching and Visualisation Parameters

Following segmentation, ViSPACE reconstructs a whole-slide segmentation map by stitching together the individual patch predictions. During this stage, publication-quality visualisation images and a GeoJSON representation of the segmentation are generated. The default stitching parameters are suitable for most applications. Users will rarely need to modify `STEP_X` or `STEP_Y`, as ViSPACE automatically determines the correct tile spacing from the segmentation manifest. The parameters most commonly adjusted are `MAX_PX`, `ALPHA`, and `THUMB_MAX`, which control the appearance and resolution of the generated visualisation images without affecting the underlying segmentation or spatial feature calculations.

2.5 Tumour ROI Extraction Parameters

Following whole-slide segmentation, ViSPACE identifies tumour-rich regions that are suitable for downstream spatial analysis. Rather than analysing every segmented tile, the pipeline groups neighbouring tumour-containing tiles into spatial clusters and generates non-overlapping square regions of interest (ROIs). These ROIs are used as the input for all subsequent tumour microenvironment analyses, including tumour–stroma ratio (TSR), stromal tumour-infiltrating lymphocyte (sTIL) quantification, immune proximity analysis, necrosis proximity analysis, and tumour morphology measurements. For most applications, the default ROI extraction parameters provide robust tumour localisation. Users may adjust `ROI_SIZE_UM`, `ROI_MIN_TUMOR_FRAC`, `ROI_MIN_CLUSTER_TILES`, and `ROI_MERGE_GAP_UM` when analysing datasets with markedly different tumour morphology or tissue architecture. The visualisation parameters `ROI_THUMB_WIDTH` and `ROI_COLOR_BY_CLUSTER` affect only the appearance of the generated overlay figures and have no impact on the computed spatial features.

2.6 Cluster TSR and sTILs Scoring Parameters

Following tumour ROI extraction, ViSPACE constructs non-overlapping cluster scoring polygons by buffering and merging the tumour ROI boxes. Within each scoring polygon, the segmented tissue is used to compute the tumour–stroma ratio (TSR) and stromal tumour-infiltrating lymphocyte (sTIL) score. Background pixels are excluded from all quantitative measurements, while tissue coverage is recorded as a quality-control metric.

The default configuration is suitable for most whole-slide image analyses and follows the recommended Salgado definition of stromal tumour-infiltrating lymphocytes. In routine applications, `TILS_DENOMINATOR` should remain set to `salgado`. The parameters most likely to require adjustment are `CLUSTER_BUFFER_UM`, which controls the extent of the scoring polygon around tumour regions, and `CLUSTER_MIN_ROI_BOXES`, which determines the minimum cluster size included in the analysis. The remaining parameters primarily act as quality-control filters and should generally retain their default values unless analysing datasets with substantially different tissue characteristics.

2.7 Immune Proximity Parameters

The immune proximity stage quantifies the spatial relationship between tumour-infiltrating lymphocytes (TILs) and the tumour boundary. Using the cluster scoring polygons generated in the previous stage, ViSPACE measures the distribution of inflammatory tissue relative to tumour regions and classifies each tumour cluster into biologically meaningful immune phenotypes. The default parameter values were selected to characterise immune organisation in H&E whole-slide images and generally require no modification. The proximity thresholds determine the spatial distances reported in the output, whereas the remaining parameters define the decision boundaries used to classify tumour clusters into the *immune-desert*, *immune-penetrated*, *margin-localised*, *immune-excluded*, and *peritumoral* immune phenotypes. These values should only be adjusted when alternative biological definitions or cohort-specific criteria are required.

2.8 Necrosis Proximity Parameters

The necrosis proximity stage characterises the spatial relationship between necrotic tissue, tumour, inflammatory tissue, and stroma within each tumour cluster. In addition to quantifying the distribution of necrosis relative to the tumour boundary, the analysis measures necrosis–immune coupling and assigns a biologically interpretable necrosis phenotype to each tumour cluster. The default values were selected to provide robust characterisation of necrotic tissue organisation in H&E whole-slide images and generally require no modification. The proximity thresholds determine the spatial distance statistics reported by the pipeline, whereas the remaining parameters define the decision boundaries used to classify tumour clusters into the *necrosis-absent*, *tumour-central*, *peritumoural*, *immune-adjacent*, and *stromal-distant* necrosis phenotypes. These parameters should only be adjusted when alternative biological definitions or dataset-specific criteria are required.

2.9 Tumour Morphology Parameters

The final stage of the ViSPACE pipeline quantifies the morphological characteristics of tumour tissue within each scored tumour cluster. Following segmentation, neighbouring tumour polygons are merged into individual tumour islands, from which a range of geometric, shape, and fragmentation features are computed. Small isolated tumour fragments can optionally be excluded to reduce the influence of segmentation noise. For routine analyses, the default parameters are recommended. The minimum tumour island area of **1000 μm^2** effectively removes very small tumour fragments that would otherwise distort fragmentation and shape metrics, while preserving biologically meaningful tumour islands. Quality-control output should only be enabled when validating a new dataset or selecting an alternative island-size threshold. The tumour class names should remain unchanged unless the segmentation model produces different class labels.

Table 2.1: Complete ViSPACE configuration reference.

Group	Parameter	Default	Use	Description
Required	`WSI_PATH`	required	**Adjustable**	Path to the input WSI, such as `.svs`, `.tif`, or another OpenSlide-compatible format.
Required	`CHECKPOINT`	required	**Adjustable**	Path to the trained ViSPACE decoder checkpoint (`.pt`).
Required	`OUT_DIR`	`vispace_output`	**Adjustable**	Root directory for all intermediate and final outputs.
Tessellation	`PATCH_SIZE`	224	**Fixed**	Patch edge length in pixels. The model was trained on 224 × 224 patches.
Tessellation	`MPP`	0.25	**Fixed**	Microns per pixel. The released weights assume 0.25 μm/pixel; this value also calibrates all area and distance features.
Tessellation	`WORKERS`	4	**Adjustable**	Number of parallel CPU workers used during Mussel tessellation.
Tessellation	`SEGMENT_THRESH`	20	**Adjustable**	Otsu-based tissue-mask threshold. Tune only for unusual staining or scanning artefacts.
Tessellation	`THUMBNAIL_SIZE`	`(1024,1024)`	**Adjustable**	Tessellation thumbnail dimensions in pixels; affects quality-control previews only.
Model	`VIRCHOW2_PATH`	`None`	**Optional**	Local Virchow2 weights. When `None`, weights are fetched from Hugging Face after authentication.
Model	`DEVICE`	`cuda`	**Fixed**	Inference device required for the standard segmenter workflow. CUDA is expected for routine use.
Model	`BATCH_SIZE`	32	**Adjustable**	Number of patches processed per inference batch. Reduce when GPU memory is insufficient.
Model	`WHITE_THRESH`	220	**Adjustable**	Near-white background threshold used to avoid inference on blank areas.
Stitching	`STEP_X`	444	**Optional**	Informational horizontal tile spacing. `stitch.py` normally infers the real step from `manifest.csv`.
Stitching	`STEP_Y`	444	**Optional**	Informational vertical tile spacing. `stitch.py` normally infers the real step from `manifest.csv`.
Stitching	`MAX_PX`	4096	**Adjustable**	Maximum long edge of generated slide-level PNG visualisations.
Stitching	`ALPHA`	0.6	**Adjustable**	Overlay blend weight; affects visualisation only.
Stitching	`THUMB_MAX`	2048	**Adjustable**	Maximum thumbnail dimension for visualisation.
Tumour ROI	`ROI_SIZE_UM`	200	**Adjustable**	Edge length of each square tumour ROI in microns.
Tumour ROI	`ROI_MIN_TUMOR_FRAC`	0.20	**Adjustable**	Minimum tumour fraction required for a tile to enter ROI clustering.
Tumour ROI	`ROI_MAX_NECROSIS`	0.50	**Adjustable**	Maximum mean necrosis fraction allowed within a candidate ROI.
Tumour ROI	`ROI_MIN_CLUSTER_TILES`	3	**Adjustable**	Minimum connected tumour-tile count required to retain a clump.
Tumour ROI	`ROI_CONNECTIVITY`	8	**Adjustable**	Grid connectivity for clustering: 4 for edge neighbours or 8 for edge and corner neighbours.
Tumour ROI	`ROI_MERGE_GAP_UM`	60	**Adjustable**	Maximum gap in microns for merging nearby tumour clumps.
Tumour ROI	`ROI_THUMB_WIDTH`	1800	**Optional**	Width of the WSI thumbnail used for ROI overlay output.
Tumour ROI	`ROI_COLOR_BY_CLUSTER`	`True`	**Optional**	Colour ROI boxes by cluster identity in overlay images.
TSR/sTILs	`TILS_DENOMINATOR`	`salgado`	**Adjustable**	Selected sTIL denominator: `salgado`, `stroma_plus_inflammatory`, or `tissue`.
TSR/sTILs	`CLUSTER_BUFFER_UM`	300	**Adjustable**	Buffer around dissolved ROI boxes before cluster scoring.
TSR/sTILs	`CLUSTER_MIN_ROI_BOXES`	5	**Adjustable**	Minimum ROI-box count required to retain a cluster.
TSR/sTILs	`CLUSTER_MIN_POLYGON_AREA_PX2`	1.0	**Optional**	Minimum GeoJSON polygon area retained during scoring.
TSR/sTILs	`CLUSTER_MAX_AREA_QUANTILE`	1.0	**Optional**	Upper polygon-area quantile filter; 1.0 disables it.
TSR/sTILs	`CLUSTER_MAX_AREA_MAD_Z`	0.0	**Optional**	MAD-based polygon-area outlier threshold; 0 disables it.
TSR/sTILs	`CLUSTER_MIN_TISSUE_FRACTION`	0.10	**Adjustable**	Minimum segmented tissue fraction required for reliable cluster scores.

Continued on next page

Table 2.1 continued from previous page

Group	**Parameter**	**Default**	**Use**	**Description**
TSR/sTILs	`CLUSTER_MIN_TSR_DENOM_PX2`	5000	**Adjustable**	Minimum tumour-plus-stroma denominator area for reliable TSR.
TSR/sTILs	`CLUSTER_MIN_TILS_DENOM_PX2`	5000	**Adjustable**	Minimum selected sTIL denominator area for reliable sTILs.
TSR/sTILs	`CLUSTER_NO_OVERLAY`	`False`	**Optional**	Skip rendering the cluster overlay when set to `True`.
Immune	`IMMUNE_PROXIMITY_THRESHOLDS_UM`	`[20,50,100, 200]`	**Adjustable**	Distances used to report the percentage of TIL area near the tumour boundary.
Immune	`IMMUNE_CONTACT_TOLERANCE_UM`	5	**Adjustable**	Maximum distance counted as tumour–immune contact.
Immune	`IMMUNE_DESERT_MAX_AREA_UM2`	1000	**Adjustable**	TIL-area threshold below which a cluster is classified as immune-desert.
Immune	`IMMUNE_PENETRATED_MIN_INTRA_FRAC`	0.30	**Adjustable**	Minimum intratumoral TIL fraction for the immune-penetrated phenotype.
Immune	`IMMUNE_MARGIN_MIN_PCT_50UM`	50	**Adjustable**	Minimum TIL percentage within 50 μm for the margin-localised phenotype.
Immune	`IMMUNE_EXCLUDED_MIN_MEDIAN_UM`	100	**Adjustable**	Minimum median extratumoral distance for the immune-excluded phenotype.
Immune	`IMMUNE_PERITUMORAL_MAX_MEDIAN_UM`	100	**Adjustable**	Maximum median extratumoral distance for the peritumoral phenotype.
Necrosis	`NECROSIS_PROXIMITY_THRESHOLDS_UM`	`[50,100]`	**Adjustable**	Distances used to report necrosis proximity to tumour.
Necrosis	`NECROSIS_CONTACT_TOLERANCE_UM`	5	**Adjustable**	Maximum distance counted as tumour–necrosis contact.
Necrosis	`NECROSIS_IMMUNE_COUPLING_ THRESHOLD_UM`	20	**Adjustable**	Maximum necrosis-to-immune distance counted as coupled.
Necrosis	`NECROSIS_MIN_COMPONENT_AREA_UM2`	500	**Adjustable**	Minimum necrotic component area retained for analysis.
Necrosis	`NECROSIS_ABSENT_MAX_AREA_UM2`	500	**Adjustable**	Area threshold below which a cluster is classified as necrosis-absent.
Necrosis	`NECROSIS_CENTRAL_MIN_INTRA_FRAC`	0.50	**Adjustable**	Minimum intratumoral necrosis fraction for the tumour-central phenotype.
Necrosis	`NECROSIS_PERITUMOURAL_MIN_PCT_ 100UM`	60	**Adjustable**	Minimum necrosis percentage within 100 μm for the peritumoural phenotype.
Necrosis	`NECROSIS_IMMUNE_ADJACENT_MIN_ COUPLING`	0.20	**Adjustable**	Minimum immune-coupling index for the immune-adjacent phenotype.
Necrosis	`NECROSIS_DISTANT_MIN_MEDIAN_UM`	150	**Adjustable**	Median-distance threshold associated with stromal-distant necrosis.
Morphology	`MORPHOLOGY_MIN_ISLAND_AREA_UM2`	1000	**Adjustable**	Minimum tumour-island area used for shape and fragmentation metrics. Report this threshold with published morphology results.
Morphology	`MORPHOLOGY_SAVE_ISLAND_QC`	`False`	**Optional**	Write `tumor_island_qc.csv` with per-island measurements.
Morphology	`MORPHOLOGY_TUMOR_CLASS_NAMES`	tumour aliases	**Optional**	Accepted labels: `Tumour`, `Tumor`, `tumour`, and `tumor`.

2.10 Create a Reusable Configuration

The recommended workflow is to create one JSON configuration and reuse it for the environment check, pipeline runner, individual stages, and report generator.

```
python config.py \
  --wsi-path slides/example.svs \
  --checkpoint TNBC_weights/TNBC_best.pt \
  --out-dir vispace_output \
  --device cuda \
  --batch-size 32 \
  --print-config > run_config.json
```

Inspect the saved configuration before starting a long run:

```
python environment_check.py --from-json run_config.json
```

Chapter 3

Running the ViSPACE Pipeline

The full pipeline is orchestrated by `run_vispace.py`. It executes stages in dependency order, skips stages whose sentinel outputs already exist, stops on the first failure, and can rerun selected stages when requested.

3.1 Complete Run

```
python run_vispace.py --from-json run_config.json
```

From Python:

```
from config import PipelineConfig
from run_vispace import run_vispace

cfg = PipelineConfig(
    WSI_PATH="slides/example.svs",
    CHECKPOINT="TNBC_weights/TNBC_best.pt",
    OUT_DIR="vispace_output",
)

results = run_vispace(cfg.WSI_PATH, cfg)
print(results["success"])
print(results["total_elapsed_s"])
```

3.2 Pipeline Command Reference

Table 3.1 is the single execution reference for the pipeline. Commands assume that `run_config.json` has already been created.

Table 3.1: ViSPACE pipeline stages and commands.

Stage	**Script or action**	**Purpose**	**Command**
–	`environment_check.py`	Validate dependencies, CUDA, package versions, paths, authentication, and permissions.	`python environment_check.py --from-json run_config.json`
1	`tessellate.py`	Tile tissue regions with Mussel.	`python tessellate.py --from-json run_config.json`
2	`segmenter.py`	Run Virchow2 and the pixel-wise decoder.	`python segmenter.py --from-json run_config.json`
3	`stitch.py`	Stitch tile predictions and generate whole-slide GeoJSON.	`python stitch.py --from-json run_config.json`
4	`tumor_roi_overlay.py`	Identify tumour-rich tiles, form clusters, and generate ROI boxes.	`python tumor_roi_overlay.py --from-json run_config.json`
5	`cluster_tils_tsr_scoring.py`	Build scoring polygons and compute cluster and WSI TSR/sTILs.	`python cluster_tils_tsr_scoring.py --from-json run_config.json`
6	`immune_proximity_features.py`	Quantify TIL position relative to tumour boundaries.	`python immune_proximity_features.py --from-json run_config.json`
7	`necrosis_proximity_features.py`	Quantify necrosis position and tissue-context coupling.	`python necrosis_proximity_features.py --from-json run_config.json`
8	`tumor_morphology_features.py`	Compute tumour area, shape, island, spread, and fragmentation features.	`python tumor_morphology_features.py --from-json run_config.json`
All	`run_vispace.py`	Execute all analytical stages in order and resume from completed outputs.	`python run_vispace.py --from-json run_config.json`
Subset	`run_vispace.py--stages`	Run selected stages after their prerequisites already exist.	`python run_vispace.py --from-json run_config.json --stages cluster_tils_tsr_score, immune_proximity,necrosis_proximity`
Force	`run_vispace.py--force-stages`	Rerun specified stages even when sentinel outputs exist.	`python run_vispace.py --from-json run_config.json --force-stages tumor_roi_overlay`
Report	`report/generate_report.py`	Generate and optionally render a slide-specific Quarto report.	`python report/generate_report.py --from-json run_config.json --render --to html`

3.3 Processing Multiple Slides

ViSPACE processes one slide per call. Use a base configuration and replace only `WSI_PATH` for each slide:

```
from pathlib import Path
from dataclasses import replace
from config import PipelineConfig
from run_vispace import run_vispace

base_cfg = PipelineConfig(
    CHECKPOINT="TNBC_weights/TNBC_best.pt",
    OUT_DIR="vispace_output",
)

for slide in Path("slides").glob("*.svs"):
    cfg = replace(base_cfg, WSI_PATH=str(slide))
    result = run_vispace(str(slide), cfg)
    if not result["success"]:
        print(f"Failed: {slide}")
```

3.4 Jupyter Notebook Use

Every stage exposes a Python function of the form `run_*(wsi_path,cfg)`. A notebook can keep one configuration object in memory and call stages directly. After editing a module during an interactive session, reload it with `importlib.reload` before rerunning the stage.

Chapter 4

Output Directory Structure

ViSPACE creates one directory per slide under `OUT_DIR`. The slide name is derived from the WSI filename stem.

```
vispace_output/
‘-- <slide_name>/
    |-- tessellation/
    |   |-- patches/
    |   |-- <slide>.h5
    |   |-- mask.png
    |   |-- grid_mask.png
    |   ‘-- thumbnail.png
    |-- segmentation/
    |   |-- manifest.csv
    |   |-- segmentation_all_classes.geojson
    |   ‘-- <slide>_segmentation.png
    |-- spatial_feature_results/
    |   |-- tumor_roi_overlay/
    |   |-- cluster_tils_tsr_score/
    |   |-- immune_proximity/
    |   |-- necrosis_proximity/
    |   ‘-- tumor_morphology/
    |-- <slide>_vispace_report.qmd
```

4.1 Output Files

Table 4.1: Principal ViSPACE output files.

Directory	File	Purpose
`tessellation`	`patches/`	Extracted 224×224 patch PNGs.
`tessellation`	`<slide>.h5`	Mussel tile manifest and tessellation metadata.
`tessellation`	`mask.png`, `grid_mask.png`, `thumbnail.png`	Tissue mask, accepted grid, and tessellation thumbnail.
`segmentation`	`manifest.csv`	Per-tile coordinates and predicted class fractions.
`segmentation`	`segmentation_all_classes.geojson`	Whole-slide tissue polygons used by spatial stages.
`segmentation`	`<slide>_segmentation.png`	Colour segmentation quality-control image.

Continued on next page

Table 4.1 continued from previous page

Directory	**File**	**Purpose**
`tumor_roi_overlay`	`tumor_roi_boxes.csv`	ROI coordinates and composition; primary input to cluster scoring.
`tumor_roi_overlay`	`tumor_roi_boxes_pseudo_thumbnail.png`	ROI overlay on the segmentation-derived pseudo-thumbnail.
`tumor_roi_overlay`	`tumor_roi_boxes_wsi_thumbnail.png`	Optional ROI overlay on the source WSI thumbnail.
`cluster_tils_tsr_score`	`cluster_scoring_polygons.geojson`	Final non-overlapping cluster scoring polygons.
`cluster_tils_tsr_score`	`tils_tsr_by_cluster.csv`	Cluster-level TSR, sTILs, tissue areas, and reliability fields.
`cluster_tils_tsr_score`	`tils_tsr_wsi_summary.csv`	Area-weighted whole-slide TSR and sTIL summary.
`cluster_tils_tsr_score`	`cluster_tils_tsr_overlay.png`	Tissue map and cluster-score visualisation.
`immune_proximity`	`immune_proximity_by_cluster.csv`	Cluster-level TIL proximity and immune phenotype features.
`immune_proximity`	`immune_proximity_wsi_summary.csv`	Whole-slide immune proximity summary.
`immune_proximity`	`immune_proximity_plot.png`	Immune distance, proximity-zone, and phenotype visualisation.
`necrosis_proximity`	`necrosis_proximity_by_cluster.csv`	Cluster-level necrosis proximity and phenotype features.
`necrosis_proximity`	`necrosis_proximity_wsi_summary.csv`	Whole-slide necrosis proximity summary.
`necrosis_proximity`	`necrosis_distance_figure.png`	Necrosis distance and proximity-zone visualisation.
`necrosis_proximity`	`necrosis_tissue_context_figure.png`	Necrosis–immune coupling and phenotype visualisation.
`tumor_morphology`	`tumor_core_features_by_cluster.csv`	Cluster-level tumour area, shape, island, spread, and fragmentation features.
`tumor_morphology`	`tumor_core_wsi_summary.csv`	Whole-slide morphology summary.
`tumor_morphology`	`tumor_island_qc.csv`	Optional per-island quality-control measurements.
`report`	`<slide>_vispace_report.qmd`	Slide-specific Quarto source.
`report`	`<slide>_vispace_report.html`	Rendered HTML report when Quarto rendering is enabled.

Chapter 5

Report Generation

Report generation is the final, optional post-processing step in the ViSPACE workflow. It should be performed only after all analytical stages have completed and their outputs have been written to the slide-specific output directory.

The report generator does not rerun tessellation, segmentation, stitching, or spatial feature extraction. Instead, it reads the completed ViSPACE outputs, inserts the current whole-slide image path and output directory into a Quarto template, and writes a slide-specific Quarto Markdown (`.qmd`) file.

The generated `.qmd` file must subsequently be rendered using the Quarto command-line interface.

5.1 Generating the QMD File in JupyterLab

Within a JupyterLab notebook, generate the slide-specific report source using the existing ViSPACE configuration object:

```
from generate_qmd_file import generate_report

report_qmd = generate_report(
wsi_path=cfg.WSI_PATH,
cfg=cfg,
)

print(f"Generated report file: {report_qmd}")
```

5.2 Rendering the Report with Quarto

After generating the `.qmd` file, render it from the Linux terminal using:

```
quarto render vispace_report_slide.qmd
```

The `quarto render` command creates the final document using the output format declared in the Quarto YAML front matter. The target `.qmd` file must be supplied as the first command-line argument.